Trabalho Prático: Orquestração de Serviços, Sistemas Distribuídos e Monitoramento com Containers

Prof. Michel Pires da Silva

Centro Federal de Educação Tecnológica de Minas Gerais — CEFET-MG

Universidade Federal de Ciência e Inovação de Minas Gerais — UFCI-MG

Departamento de Computação (DECOM)

Valor: 20 Pontos | **Entrega:** 21/11/2026 | **Apresentações:** 23/11/2026

---

1 Apresentação Geral e Objetivos

O projeto prático tem como objetivo trazer os conceitos teóricos para o mundo real da engenharia de software e infraestrutura de TI. Ao invés de executar aplicações isoladas, as arquiteturas associadas à sistemas operacionais modernos dividem serviços em múltiplos componentes (microsserviços) que precisam se comunicar, compartilhar armazenamento e serem monitorados continuamente. Neste trabalho, os grupos utilizarão ferramentas de orquestração (como **Docker Compose** ou **Kubernetes**) para planejar, implantar e validar ecossistemas com múltiplos containers trabalhando em conjunto para formar uma solução completa (ex: servidores web, proxies, bancos de dados e painéis de monitoramento).

2 Práticas Sugeridas

A turma será organizada em equipes. Cada equipe deverá escolher uma das arquiteturas abaixo. Todas as trilhas exigem a criação de arquivos de manifesto/configuração (*docker-compose.yml* do K8s) e a demonstração prática de funcionamento do sistema sob condições específicas.

Prática 1 — Infraestrutura Web e Balanceamento de Carga

- **Cenário:** Quando um site tem muitos acessos, um único servidor não dá conta. Usa-se um "Proxy Reverso" para receber as requisições e distribuí-las entre várias cópias da aplicação, que por sua vez acessam um banco de dados centralizado.
- **O que deve ser feito (Roteiro Prático):**
  1. Criar um `docker-compose.yml` que suba 4 containers de forma integrada:
     - **1 Proxy Reverso:** Pode ser o *Nginx* ou *HAProxy*.
     - **2 Instâncias da Aplicação:** Uma API simples feita pelo grupo (em Python/Flask, Node.js, Java, etc.) que se conecte ao banco e retorne qual container processou a requisição (ex: retornando o hostname).

  - **1 Banco de Dados:** PostgreSQL, MySQL ou MongoDB.
2. Configurar uma **Rede Virtual (Docker Network)** para que os containers se comuniquem, mas apenas o Proxy tenha a porta exposta para o mundo externo.
3. Configurar um **Volume (Docker Volume)** no banco de dados para garantir persistência.

- **Validação para a Apresentação:**
  - Fazer requisições ao sistema no navegador e mostrar que a resposta alterna entre a "Aplicação 1" e "Aplicação 2" (comprovando o balanceamento de carga). Para esse ponto, avalie os diferentes escalonamentos disponíveis, tendo como *baseline* o *Round-Robin*.
  - Derrubar o container do banco de dados (`docker stop`), iniciar novamente e demonstrar que os dados salvos não foram perdidos, comprovando o isolamento de armazenamento (Volumes).

Prática 2 — Observabilidade e Monitoramento de Recursos do SO

- **Cenário:** Em um ambiente de produção, é crítico saber se um servidor está sobrecarregado (falta de CPU, RAM estourando). O grupo montará um painel de controle profissional para monitorar métricas do Sistema Operacional hospedeiro e dos containers.
- **O que deve ser feito (Roteiro Prático):**
1. Criar um `docker-compose.yml` que suba uma *Stack* de monitoramento contendo:
   - **Prometheus:** Banco de dados de séries temporais que coleta e armazena as métricas.
   - **Grafana:** Ferramenta de interface gráfica para criação de Dashboards interativos.
   - **cAdvisor** (monitora os containers) e/ou **Node Exporter** (monitora o SO hospedeiro).
2. Configurar o Prometheus para buscar os dados (fazer *scraping*) dos agentes coletores.
3. Acessar o Grafana e montar (ou importar) um painel visual (Dashboard) com gráficos de uso de CPU, Memória, I/O de disco e tráfego de Rede.

- **Validação para a Apresentação:**
  - Com o painel do Grafana aberto, rodar no computador hospedeiro (ou em um container extra) uma ferramenta de estresse como o `stress-ng` ou rodar um script em *loop infinito*.
  - Mostrar, em tempo real, os gráficos reagindo ao pico de processamento/memória, relacionando a visualização gráfica com o conceito de contenção de recursos do SO.

Prática 3 — Central de Logs, Auditoria e Segurança (Stack ELK)

- **Cenário:** Analisar logs espalhados por dezenas de servidores em arquivos de texto soltos (`/var/log`) é impossível. O mercado utiliza *pipelines* de centralização de logs para cruzar dados e gerar alertas de segurança.
- **O que deve ser feito (Roteiro Prático):**
  1. Subir um serviço web qualquer (ex: um servidor *Nginx* simples ou Apache).
  2. Criar um `docker-compose.yml` para subir a Stack ELK clássica:
     - **Elasticsearch:** Motor de busca e armazenamento dos logs.
     - **Logstash / Filebeat:** Coletor que lê os arquivos de log do servidor Nginx e os envia para o Elasticsearch.
     - **Kibana:** Painel web para buscar e visualizar os logs.
  3. Configurar o Filebeat para ler os logs de acesso (`access.log`) e de erros (`error.log`) do servidor web, transportando os dados via rede interna do Docker.
- **Validação para a Apresentação:**
  - Acessar a aplicação web normalmente para gerar tráfego limpo.
  - Simular um "ataque" ou anomalia: usar uma ferramenta como `curl` ou `dirb` para tentar acessar centenas de páginas que não existem, gerando uma avalanche de erros `HTTP 404`.
  - Abrir o Kibana e mostrar o filtro/pesquisa encontrando o momento exato do ataque, listando os IPs atacantes e a quantidade de falhas, evidenciando o poder da auditoria centralizada.

Prática 4 — Sistemas Distribuídos Assíncronos e Filas de Mensagens

- **Cenário:** Se um sistema de vendas recebe 10.000 requisições simultâneas para gerar boletos, o servidor e o banco de dados cairão. Para evitar isso, usa-se o padrão Produtor-Consumidor com uma Fila de Mensagens intermediando a comunicação (*Buffer*).
- **O que deve ser feito (Roteiro Prático):**
  1. Criar um `docker-compose.yml` contendo:
     - **RabbitMQ** (ou *Redis / Kafka*): O intermediário (Broker) que gerencia a fila.
     - **Aplicação Produtora (Producer):** Um script (ex: Python) que gera mensagens/requisições fictícias e as joga na fila o mais rápido possível.
     - **Aplicação Consumidora (Consumer):** Um script (*Worker*) que lê a fila, simula um processamento demorado (ex: `sleep(1)` segundo) e salva o resultado no banco.
     - **Banco de Dados** para salvar o resultado final.
- **Validação para a Apresentação:**

  - Rodar o script Produtor para despachar 5.000 “tarefas” instantaneamente. A aplicação não deve travar.
  - Mostrar no painel administrativo do RabbitMQ (interface web) a fila cheia.
  - Ligar 1 container Consumidor e mostrar a fila caindo devagar.
  - Dinamicamente, subir mais 3 cópias do container Consumidor (`docker-compose up -scale consumer=4`) e provar que a fila começa a esvaziar 4 vezes mais rápido, demonstrando concorrência e escalabilidade horizontal.

Prática 5 — Orquestração Avançada e Alta Disponibilidade (Kubernetes)

- **Cenário:** O Docker Compose é excelente para um único servidor hospedeiro. Porém, grandes empresas rodam sistemas em “*Clusters*” (várias máquinas agrupadas). O Kubernetes entra para gerenciar dezenas de containers distribuídos, garantindo escalabilidade automática e autorrecuperação (*Self-Healing*) caso uma aplicação ou servidor trave.
- **O que deve ser feito (Roteiro Prático):**
  1. Instalar e iniciar um cluster Kubernetes local voltado para desenvolvimento (sugestão: **Minikube**, **KinD** ou **K3s**).
  2. Criar os arquivos de manifesto (`.yaml`) da infraestrutura contendo:
     - **Deployment:** Responsável por manter a aplicação no ar. Deve ser configurado para subir uma aplicação web (ex: uma API simples ou servidor Nginx) mantendo pelo menos 3 cópias simultâneas rodando (`replicas: 3`).
     - **Service:** Configurar o balanceamento de carga interno do Kubernetes e expor a aplicação para acesso no navegador (utilizando `NodePort` ou `LoadBalancer`).
- **Validação para a Apresentação:**
  - Acessar a aplicação no navegador via porta do *Service* e provar que o sistema está respondendo corretamente.
  - **Teste de Autorrecuperação (*Self-Healing*):** No terminal, deletar manualmente um dos "Pods" que está rodando (usando `kubectl delete pod <nome-do-pod>`) simulando uma falha grave na aplicação. Mostrar em tempo real o orquestrador detectando a queda e subindo um container substituto instantaneamente para manter as 3 réplicas exigidas.
  - **Teste de Escalabilidade:** Digitar o comando para escalar a aplicação de 3 para 10 instâncias ao vivo (`kubectl scale deployment <nome> -replicas=10`) e demonstrar os novos recursos sendo alocados rapidamente pelo *cluster*.

3 Estrutura Obrigatória do Relatório Técnico

O relatório final deverá ser redigido em LaTeX, contendo:

1. **Introdução:** Apresentação do problema e da arquitetura escolhida.

2. **Mapeamento da Solução e Arquitetura:** Diagrama dos containers, redes (networks) e armazenamento (volumes) configurados.
3. **Tecnologias Utilizadas:** Descrição técnica do papel de cada container/pod no ecossistema (proxy, mensageria, banco de séries temporais, orquestrador, etc.).
4. **Execução e Validação Experimental:** Passo a passo com evidências (prints de tela e logs) demonstrando que os critérios de validação da trilha foram cumpridos com sucesso.
5. **Conclusão:** O aprendizado da equipe sobre a gestão de sistemas interconectados.
6. **Repositório:** Link obrigatório (GitHub) com os `YAMLs` de configuração (`docker-compose` ou manifestos `K8s`) e códigos fonte das aplicações (se houver).

4 Critérios de Avaliação

A avaliação totaliza **10,0 pontos**, focada na integração prática e entendimento sistêmico:

Critério	Descrição do Foco Avaliativo	Pontos
1. Configuração do Orquestrador	Correta estruturação dos manifestos (redes, volumes, réplicas, variáveis de ambiente e dependências).	6,0
2. Execução da Validação (Apresentação)	Cumprimento com êxito do roteiro de validação ao vivo ou em vídeo sem cortes (ex: simulação de falha, self-healing).	6,0
3. Qualidade do Relatório (LaTeX)	Organização lógica, diagramas arquiteturais claros e documentação robusta dos testes realizados.	5,0
4. Arguição Técnica	Domínio técnico e segurança dos membros do grupo ao explicar o fluxo de dados entre os componentes.	3,0
Total		**20** Pontos

5 Dinâmica e Diretrizes da Apresentação Prática

A finalização deste trabalho se dará através de uma sessão interativa de *Live Demo* (demonstração ao vivo) perante o professor e a turma. O objetivo não é apresentar um seminário teórico lendo slides, mas sim agir como engenheiros apresentando uma infraestrutura de software funcionando no "mundo real" em modelo de treinamento para as novas equipes.

Cada equipe disporá de um **tempo máximo de 40 minutos** para conduzir a sua apresentação. Para otimizar o tempo e garantir a atenção da audiência, a equipe deverá estruturar a sua prática na seguinte ordem:

1. **Alinhamento Arquitetural (5 a 10 min):**

- Apresentação visual da arquitetura implementada (através de diagramas mostrando os containers, redes e volumes).
- Explicação sucinta sobre o papel de cada serviço no arquivo `docker-compose.yml` ou nos manifestos do Kubernetes.

2. **Subida do Ambiente / *Deploy* (5 min):**
   - Execução ao vivo do comando de inicialização (`docker-compose up` ou `kubectl apply`).
   - Breve explicação do console/logs mostrando a infraestrutura ficando pronta para uso.
   - *Atenção:* Não é necessário gastar tempo instalando ferramentas bases (Docker, K8s) ao vivo. A infraestrutura base já deve estar previamente configurada e funcionando no notebook da equipe antes do início do tempo.
3. **Execução da Prática e Simulação de Cenários (15 a 20 min):**
   - Este é o ponto focal da apresentação. A equipe deve executar exatamente os passos previstos na subseção **"Validação para a Apresentação"** correspondente à sua prática.
   - A demonstração deve ser conduzida de forma fluida. Exemplo: se for simular uma carga no sistema, mostre o sistema ocioso primeiro, injete a carga e, em seguida, abra o monitor/dashboard para evidenciar o efeito para a turma em tempo real.
   - Se o roteiro exigir que um banco de dados seja derrubado ou que um *Pod* do Kubernetes seja deletado, realize o procedimento ao vivo e explique o comportamento do sistema durante a recuperação.
4. **Espaço para Arguição e Dúvidas (5 a 10 min):**
   - Encerramento da demonstração e abertura para perguntas por parte do professor e da turma.
   - Durante esta etapa, o professor poderá solicitar que algum membro do grupo altere uma configuração simples no código ao vivo ou execute um teste extra no terminal. **Todos os membros da equipe devem ter domínio prático do que foi construído e estar aptos a operar o terminal.**

Dica Prática: Planejem o tempo com margem de segurança. Problemas ao vivo (o famoso *efeito demonstração*) acontecem. Se um comando falhar, não entrem em pânico: a forma como a equipe depura o erro (*troubleshooting*) inspecionando os logs do terminal e corrigindo o problema na hora conta pontos valiosos a favor da maturidade técnica do grupo.